CHOI JUNGWON · ANDROID ENGINEER

차량이 보내는 신호를 앱이 이해하게 만듭니다.



2022년부터 인포카 안드로이드.
2인 안드로이드 팀에서 OBD2·블루투스 통신 파싱부터 결제·광고·안드로이드 오토까지 맡고 있습니다.

4년 1개월
안드로이드 실무 · 2022.08~

22개
지원 로케일 · B2C/B2B 2개 앱

4개
주요 프로젝트

SCROLL

INTRO

인포카는 OBD2 스캐너로 차량 상태를 읽는 진단 앱으로, 누적 1,000만 다운로드에 22개 로케일을 지원합니다. 저는 이 앱의 안드로이드를 맡아 블루투스 통신 레이어부터 결제·광고·안드로이드 오토까지 다룹니다.

소속 · 직책	**(주) 인포카 · 전임 연구원**	infoCar Inc. · 주임 → 전임
기간	**4년 1개월**	2022.08 — 재직중
서비스	**B2C · B2B**	인포카 · 인포카 비즈
주 도메인	**OBD2 · 블루투스**	안드로이드 오토 · 광고
학력	**신한대학교**	컴퓨터공학과 졸업

통신 레이어

ELM327 블루투스 연결·재연결, OBD2/UDS 응답 파싱, PID 스케줄링.

레거시 현대화

Java MVC를 Kotlin·MVVM+Hilt로 화면 단위 이관. 진단 탭·메인 메뉴는 Compose로.

AI 워크플로우

사내 위키·앱 코드를 함께 참조시켜 CS 원인 분석과 코드 감사를 자동화.

운영 · 유지보수

Crashlytics 상위 이슈 triage, Play 정책 대응(targetSdk·오토 심사), 고객 로그 분석.

협업 · 소통

PM·디자이너·iOS와 스크럼. 기술 제약 공유·조율.

한 회사, 한 서비스.
차계부 화면에서 통신 레이어까지.

2022.08 입사. 정비·세차 입력 화면 하나로 시작해 지금은 OBD2 통신 파싱·결제·광고·안드로이드 오토를 맡고 있습니다. 2인 안드로이드 팀에서 누적 5,886커밋, 2023년 이후 매년 1,100건 이상.

2026.07 — 현재 · 운영 중

CS 자동화 칸반보드 (cs-kanban)

하이웍스 크롤링 · Claude 헤드리스 AI 분석 · 답변초안 생성·원클릭 발송. 사내 서버 상시 운영, 팀 공용 도입.

2026.01 — 현재 · 진행 중

레거시 현대화 (MVC → Kotlin)

동작하는 레거시는 보존하고, 신규·수정 기능부터 Kotlin·MVVM 구조로 점진 전환.

2025.09 — 2025.12

프리셋 대시보드 (OBD 차량 데이터)

B2B 기능을 B2C에 이식 · 5종 엔진 × 4종 대시보드 · 설정 자동 복구 + 데이터 마이그레이션.

2025.03 — 2025.05

광고 미디에이션 전환

단일 AdMob → 5개 네트워크 워터폴 미디에이션 · Remote Config로 스토어 재배포 없이 광고 정책 제어 (미디에이션은 2025.12 철수).

2024.10 — 2025.02

안드로이드 오토 플랫폼 확장

Car App Library 신규 개발 · 앱-오토 화면 동기화 · 앱 종료 후 단독 동작 · 상태별 안내 화면.

2024.06 — 2024.08

본앱 사용성 리뉴얼

차량 등록 방식 리뉴얼 · 다국어 UX 통일 · 주유소·세차장·주차장 검색 · 영수증 API 리팩토링.

2023.07 — 2023.10

앱 핵심 기능 운영

블루투스 통신 · 푸시 · 알림 센터 · 정비 항목 UX/UI 2차 개발 · 차계부+정비 내부 테스트.

2022.10 — 2023.06

차계부 시스템 DB 리뉴얼

단일 테이블 → 정규화 다중 테이블 · 데이터 손실 없이 마이그레이션 · 구글 맵 위치 기반 · 다단위 글로벌.

2022.08 — 2022.09 · 입사

터미널 프로그램 · OBD 통신 기초

블루투스 connection · 소켓 통신 · UI · 로그 저장. OBD 통신 기초 학습기.

STACK

현업 4년의 도구들.

직접 다뤄온 언어·아키텍처·도구. 초록 칩은 주력 기술입니다.

언어 · 아키텍처

신규는 Kotlin, 레거시 Java 병존 · Hilt 생성자 주입

Kotlin | Java | Hilt | MVVM | Orbit MVI | Coroutines / Flow | RxJava3 | 레포지토리 패턴 | KSP

UI · 디바이스

Jetpack Compose XML 레이아웃 Car App Library Jetpack Navigation 게이지·차트 UI

머티리얼 3

데이터 · 통신

암호화 로컬 DB · 인터셉터 기반 통신 · 원격 제어

Room + SQLCipher DataStore SharedPreferences Retrofit / OkHttp

Firebase (RC · FCM · Crashlytics) WorkManager Play Billing OBD2 / UDS StateFlow

LiveData

AI 활용 · 협업

코드 생성보다 원인 분석·감사에 붙여 씀

Claude Code Git 슬랙 노션 · 피그마 스크럼

광고 · 미디에이션

PROJECT 03 · 5개 네트워크 워터폴 구축·운영 후 2025.12 철수 · 현재는 AdMob 단일 + 원격 제어

AdMob UMP (GDPR/CCPA) AppLovin ironSource Mintegral Pangle Unity Ads

HOW I WORK

AI는 코드 생성기가 아니라 분석·검증 도구로 씁니다.

사내 위키의 제품 정책과 실제 앱 코드를 함께 참조시켜 원인을 좁히고, 결론은 추론으로 끝내지 않고 실기기 측정으로 확인합니다.

01

근거 수집

사내 위키 정책 + 앱 코드
추측 대신 출처 확인

02

재현

고객 로그로 조건 특정
리플레이 테스트로 고정

03

구현

AI 페어로 작성
기존 패턴·컨벤션 준수

04

측정

실기기에서 확인
정적 분석 통과 ≠ 동작

확인되지 않은 가정은 다음 단계로 넘기지 않습니다

지식 그물망

옵시디언

도메인 용어(OBD2·ECU)·아키텍처·코딩 규칙을 노트로 엮은 지식 그물망. 링크된 맥락을 AI가 근거로 참조.

기획 · 요구사항

Notion MCP

기획서·작업 맥락·우선순위를 AI가 직접 참조.

디자인

Figma MCP

디자인 화면·UI 스펙·치수·컴포넌트를 AI가 직접 참조.

기존 · AS-IS

MVC (Java)

화면 코드가 로직·데이터를 한꺼번에. 한 곳을 고치면 영향 범위 파악이 어렵고, 상태가 흩어져 테스트도 어려움.

→

목표 · TO-BE

Kotlin · MVVM

화면은 상태만 그리고 로직·데이터는 분리. 상태 흐름이 한 방향으로 명확해 변경 영향이 잘 보임.

신규 화면은 MVVM으로 시작하고, 레거시와 만나는 지점은 어댑터로 잇습니다.

WORK

최신순 4개.

CS 자동화 · 차량 데이터 프리셋 · 광고 미디에이션 · 안드로이드 오토.

P.01 · 2026.07 — 운영 중

CS 자동화 칸반보드

하이웍스 크롤링 → AI 분석 → 초안 → 발송까지 한 파이프라인.

P.02 · 2025.09 — 12

프리셋 대시보드 (OBD)

B2B 기능의 B2C 이식 · 5종 엔진 × 4종 대시보드.

P.03 · 2025.03 — 05

광고 미디에이션 전환

단일 AdMob → 5개 네트워크 워터폴 · Remote Config.

P.04 · 2024.10 — 2025.02

안드로이드 오토 확장

Car App Library 신규 · 앱-오토 화면 동기화.

P.01 · CS 자동화 칸반보드 · CS-KANBAN

출근 전에,
CS 분석이 끝나 있도록.

매일 아침 개발자들이 하이웍스 공용 메일함을 열어 문의 확인 → 로그 zip 다운로드 → 대조 분석 → 필요하면 IDE로 코드 추적까지 반복했습니다. 이 수동 루틴 전체를 자동화하기 위해 수집부터 AI 진단·답변초안·발송까지 이어지는 파이프라인을 설계하고, 제가 맡은 범위를 구축해 팀 업무에 연결했습니다.

단계	Before · 수동	After · 파이프라인	효과
문의 확인	메일함 수동 확인	20분 주기 자동 수집 · 칸반 카드화	무인화
로그 분석	zip 수동 다운로드·대조	자동 해제·파싱 · 개인정보 마스킹	자동
원인 진단	IDE 열어 코드 추적	AI가 사내 위키·앱 코드 교차검증	자동
답변 작성	백지에서 작성	근거 인용된 한국어 초안 자동 생성	검토만
발송	초안 복사 → 수동 발송	보드에서 원클릭 발송 · 이중 발송 방지	원클릭

시스템 아키텍처: 문의 수집부터 검토·발송까지



01 · 수집

메일함 자동 수집

DOM 스크래핑 대신 응답 포맷을 분석해 API로 직접 수집. 대화 단위 시간순 판별로 미답변 문의를 가려냅니다. 개발자·고객문의·Biz 세 공용 메일함을 함 단위로 나눠 수집하고, 답장도 해당 함 명의로 나갑니다.

02 · 분석

벡터 DB 없는 Agentic RAG

직접 구축한 사내 위키(진단 규율·정책)를 Claude 헤드리스가 검색하고, 앱 코드 레포와 교차검증. 근거는 페이지 단위로 인용되고, 지식 갱신은 git pull이 전부입니다.

03 · 운영

사내 서버 무인 가동

테일넷 Ubuntu 서버에 systemd 상시 서비스로 배포. 수집→분석→초안이 무인으로 이어지고, 사람은 초안을 검토·수정해 발송합니다. 발송은 보드에서 고른 담당자의 하이웍스 계정 세션으로 나가고, 확인이 필요한 건은 카드별 스레드로 묻고 답합니다. 이 스레드를 슬랙 #cs 스레드와 양방향으로 잇는 브리지도 붙였습니다.

04 · 개선

교정 노트로 위키 자가개선

담당자가 AI 초안을 고쳐 보낼 때 남긴 이유를 학습 신호로 모읍니다. 보드에서 실행하면 서버가 위키 수정 브랜치를 만들고, 검증 게이트를 통과한 편집만 main에 자동 머지하고 분석용 저장소를 갱신해 이후 분석에 반영합니다.

WHY

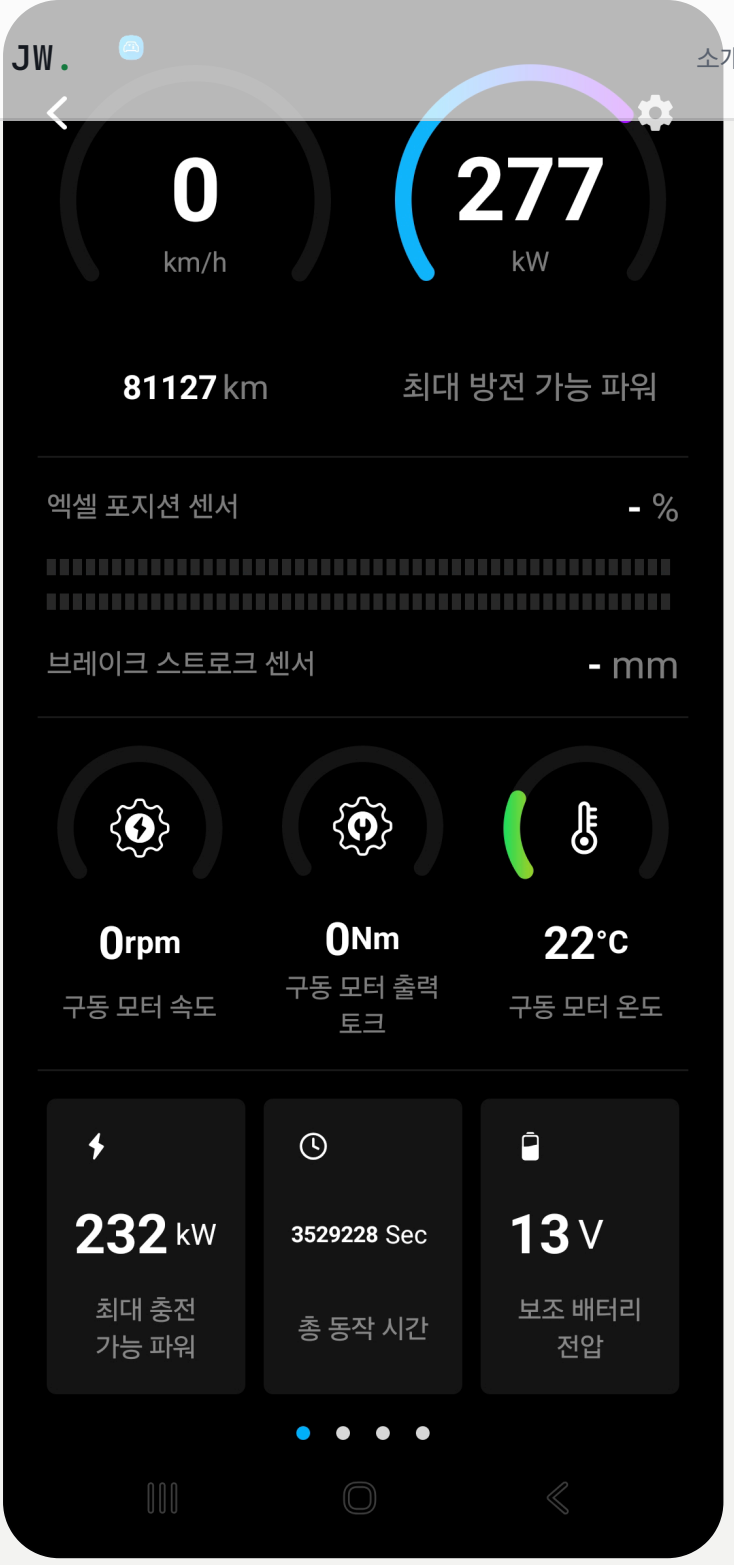
앱 코드 밖의 문제도 제가 풉니다.

CS 문의를 확인하느라 개발자 시간이 매일 새어나간다는 걸 문제로 정의하고, 수집 크롤러 · AI 분석 파이프라인 · 사내 서버 운영까지 직접 붙였습니다. 팀 공용 저장소라 혼자 만든 것은 아니고, 잡 동시성 구조 재설계와 전수 감사 · 서버 배포를 맡았습니다. 설계 결정은 ADR 문서 54건으로 기록돼 있습니다.

P.02 · 프리셋 대시보드 (OBD)

차량마다 필요한 데이터만.

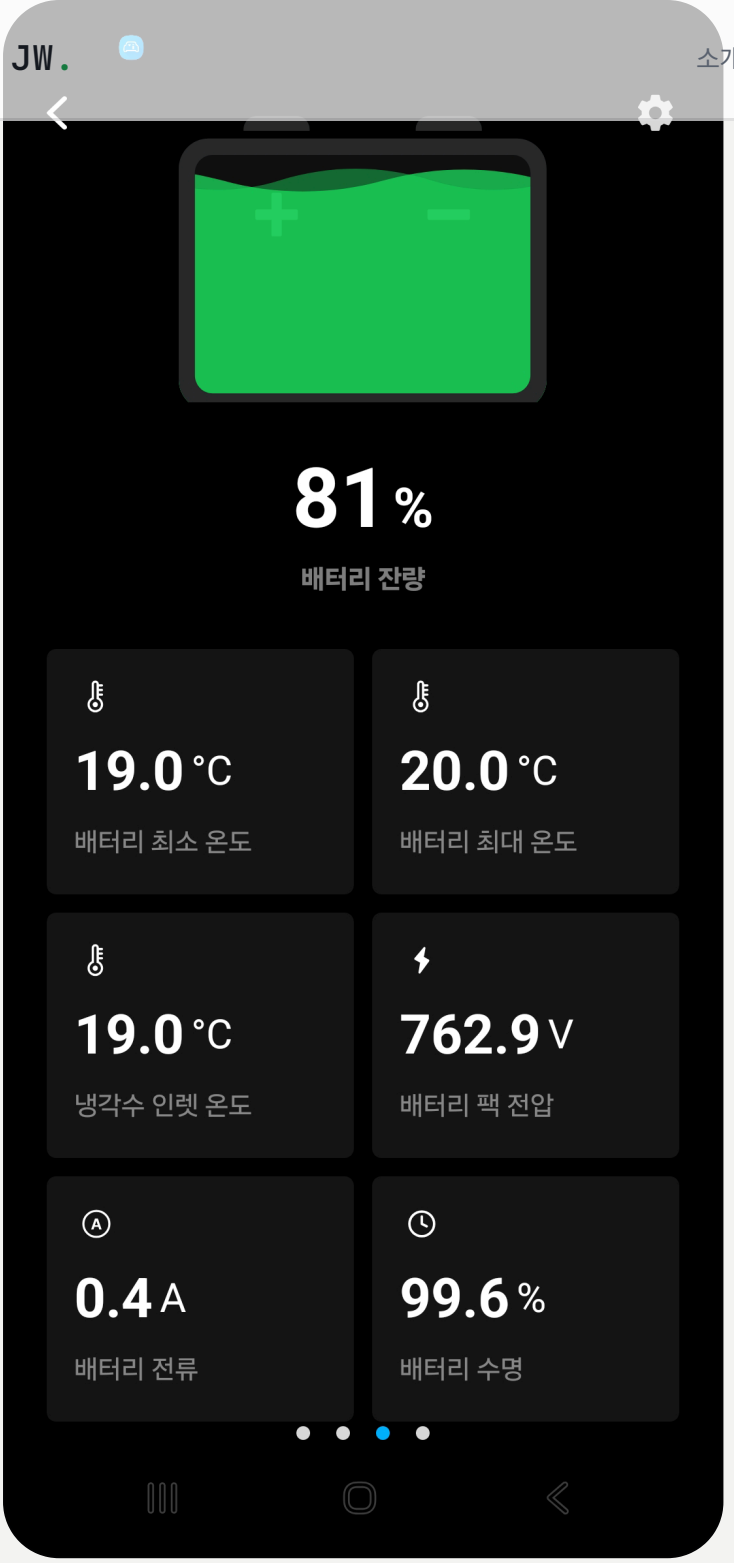
B2B(인포카 비즈) 자산을 B2C에 이식·확장하면서, 하드코딩된 센서 매핑을 사용자 선택형 + 엔진 타입별 5종 프리셋으로 재설계.



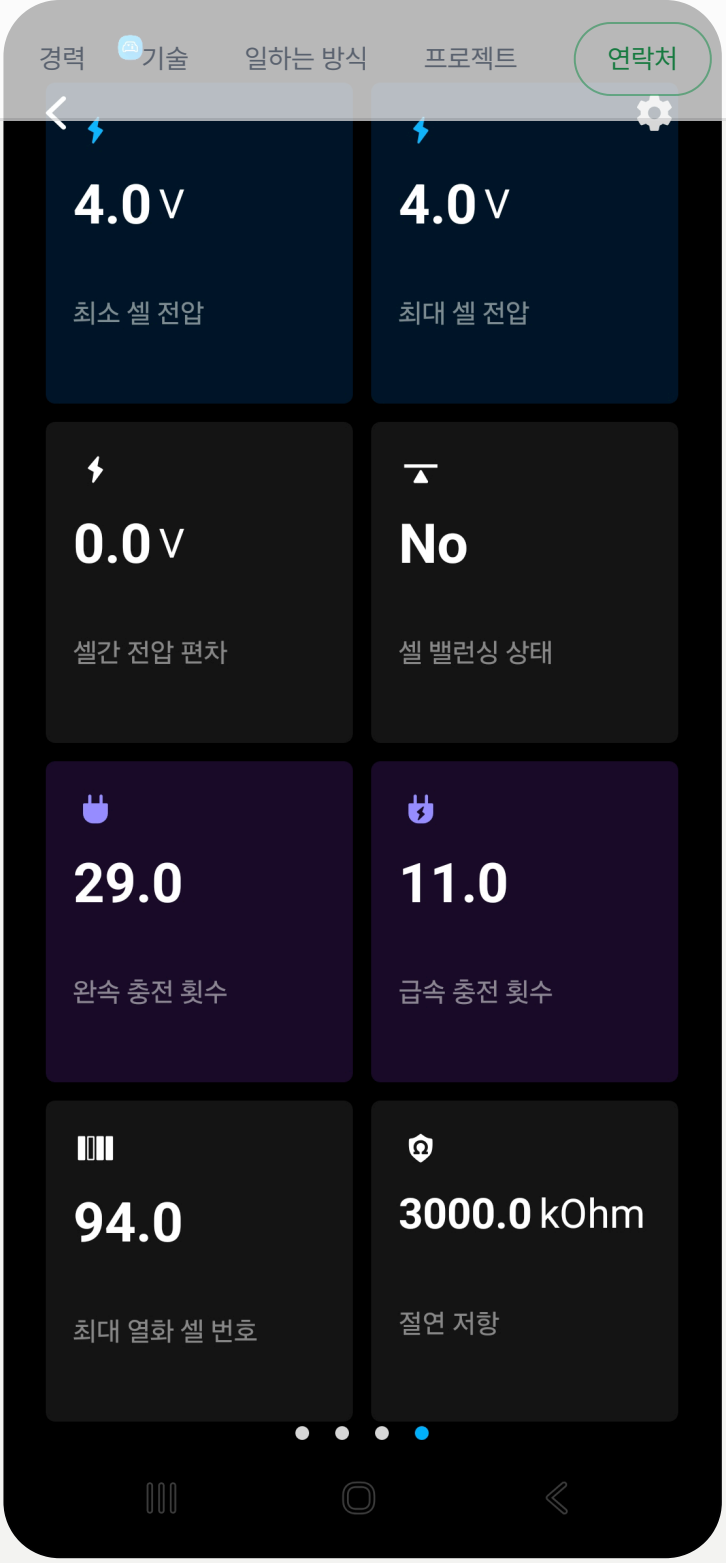
주행 · 속도·RPM·온도



TPMS · 4륜 압력·온도



배터리 · SOC·전압·SOH



서브 배터리 · 셀 편차

지표	Before	After	효과
신규 항목 추가	서너 군데 수정	한 곳만 수정	간소화
지원 엔진 타입	2종	5종	확장
레이아웃 분기	1종	4종	확장

P.03 · 광고 미디에이션 전환

배포 없이, 실시간으로 광고 정책을.

단일 AdMob에서 5개 네트워크 워터폴 미디에이션으로 확장하고, Firebase Remote Config로 스토어 재배포 없이 광고 정책을 바꿀 수 있게 했습니다. 미디에이션은 2025.12 사업 판단으로 철수했고, 원격 제어 구조는 그대로 운영 중입니다.

정책 변경 경로

무배포

스토어 재배포 → Remote Config 즉시 반영

미디에이션 운영

~2025.12

5개 네트워크 구축·운영 후 철수 · 현재 AdMob 단일

담당 범위: 안드로이드 SDK 통합 · 미디에이션 매니저 구조 · UMP 동의 자동화 · Remote Config 정책 제어. 네트워크 선정과 철수 시점은 사업 결정.

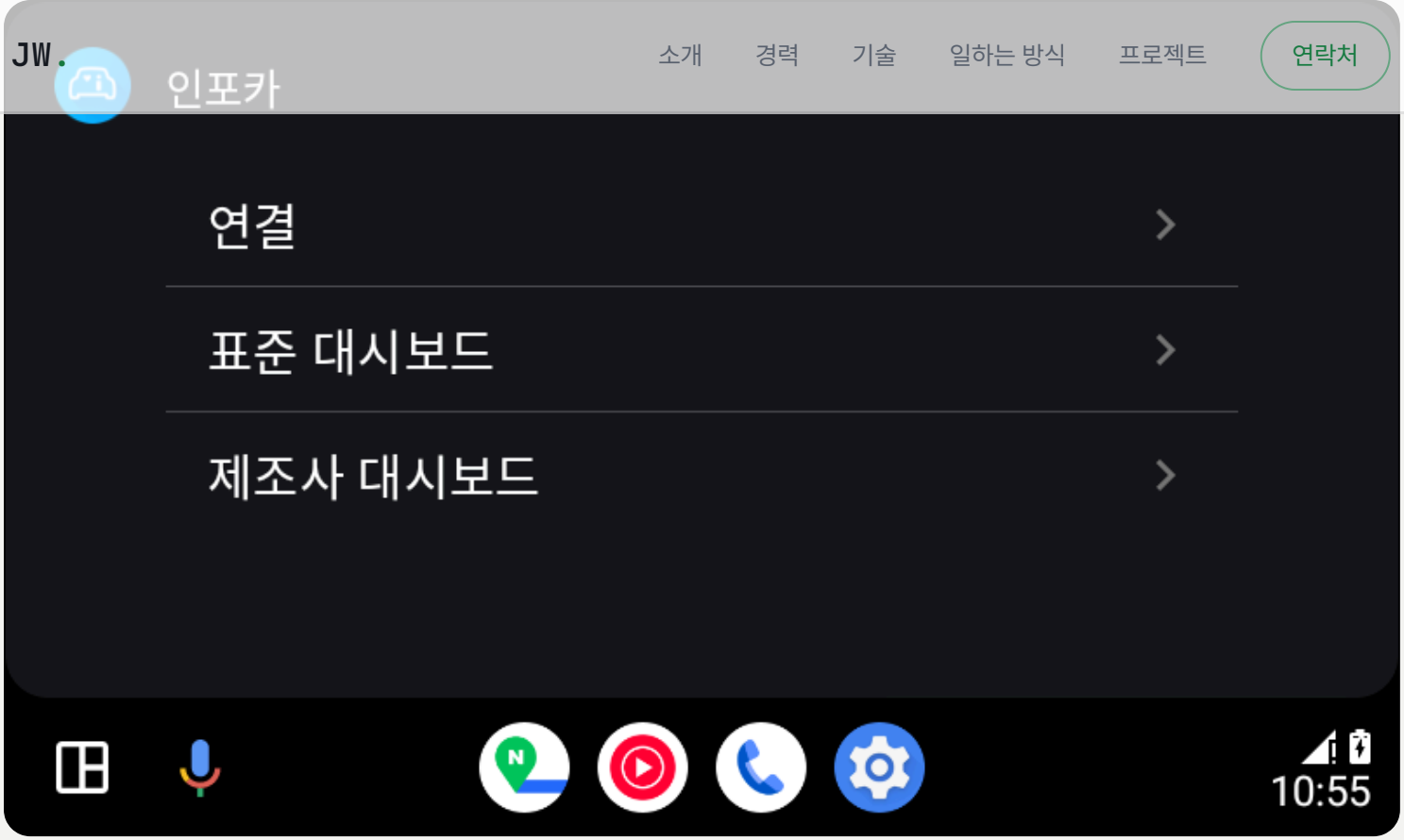
최우선 **AppLovin** › 폴백 1 **ironSource** › 폴백 2 **Mintegral** › 폴백 3 **Pangle** › 폴백 4 **Unity Ads**

지표	Before	After	효과
미디에이션 네트워크	AdMob 1개	AdMob + 5개	워터폴
광고 단위 ID	~7개	12개	세분화
정책 변경 속도	스토어 재배포	Remote Config 실시간	즉시
한국·일본 광고	없음	자체 + AdMob 통합	신규

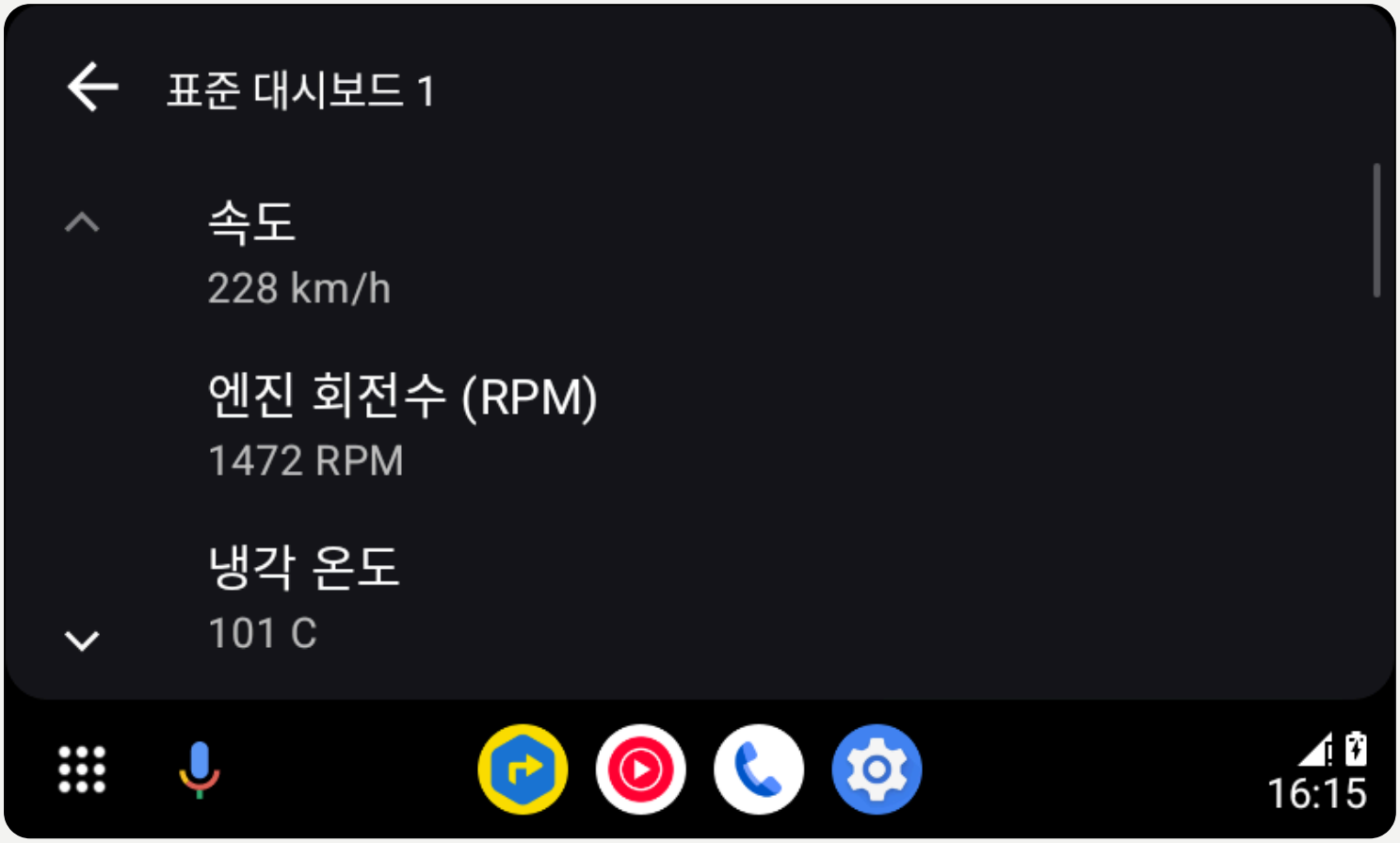
P.04 · 안드로이드 오토 플랫폼 확장

앱을 꺼도, 오토는 계속.

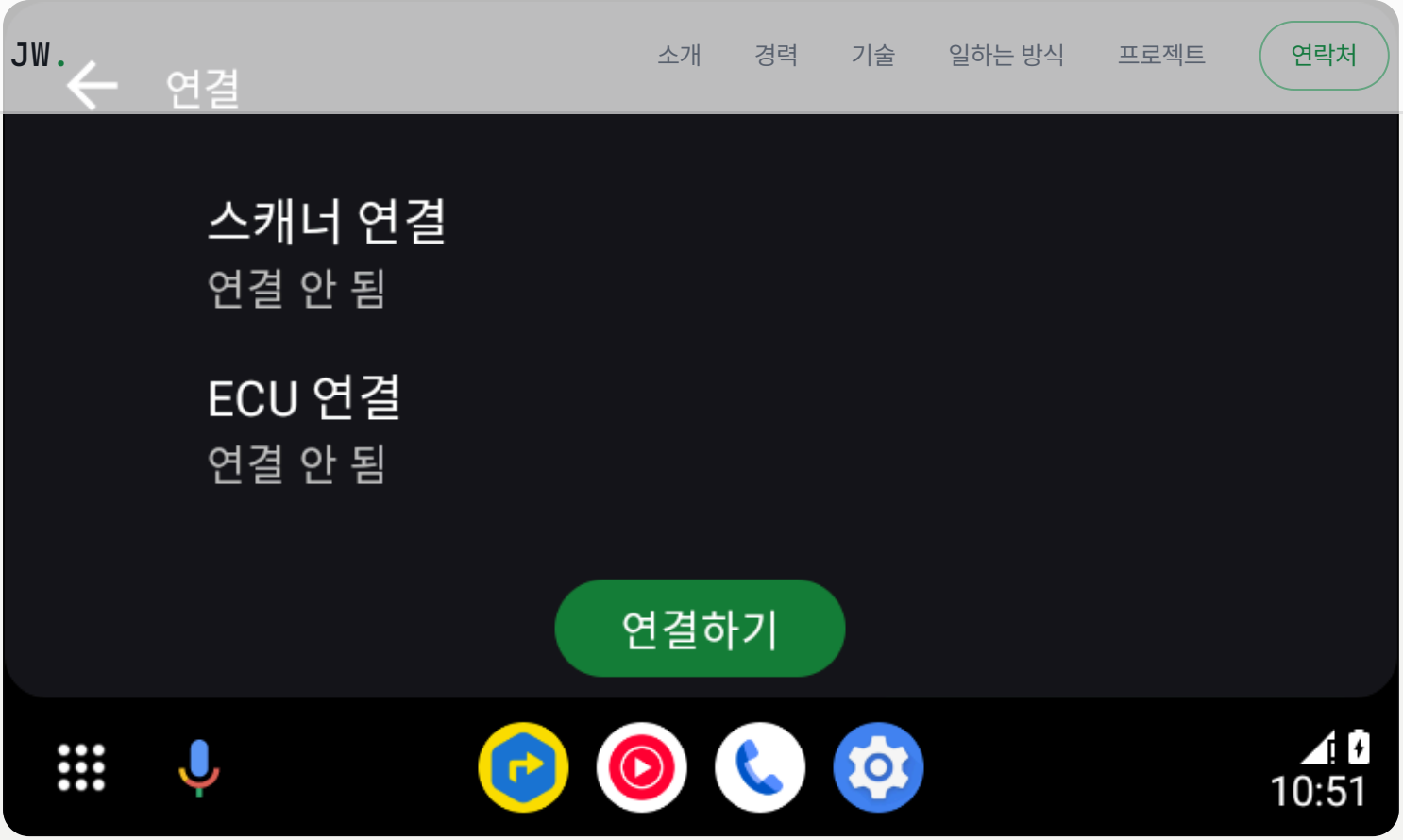
Car App Library로 신규 안드로이드 오토 앱 개발. 앱과 오토 화면을 동기화하고, 앱을 종료해도 오토가 단독으로 동작하도록 구성. 화면은 메인·연결·대시보드·표준·안내 5종.



메인 · 연결·표준·제조사 대시보드



표준 대시보드 · 실시간 매개변수



연결 · 스캐너·ECU 상태

정책 제약 인식: 안드로이드 오토는 운전 중 안전을 위해 앱 카테고리별로 쓸 수 있는 템플릿이 제한됩니다. 이 앱은 리스트·페인·메시지 3종으로 구성했고, 폰에서 쓰던 그래픽 게이지는 오토 화면으로 옮기지 않았습니다. → 화면당 정보량을 줄이고, 권한·연결·구매·다운로드 상황별 안내 페이지 12종을 따로 만들어 대응.

연결 상태 머신 (0–3)

OBD·ECU 두 채널의 상태를 코드로 명확히. 둘 다 2일 때만 데이터 표시.

0	미연결	→ 안내 스크린
1	연결 중	→ 스피너 표시
2	**연결 완료**	**→ 데이터 표시**
3	실패	→ 오류 안내

CONTACT

읽어주셔서 감사합니다.

다음 4년을 함께할 팀을 찾고 있습니다.

연락처
010-9376-2966

이메일
dkwkrhrh07@naver.com

깃허브
github.com/JayChoi07